



Supply Chain Security

Understanding the impact and how to implement practical defense

Julien Virey (`@BarbUk`)

DevSecOps @ OceanDBA

`$ whoami` Julien Virey (`@BarbUk`)

- **DevSecOps** at **OceanDBA**
- **Open-Source Contributor & Package Maintainer**
  - Maintainer of > **100 Arch Linux (AUR) packages** (aur.archlinux.org)
  - Creator of CLI utilities (`snippy`), and contributor on multiple Linux tooling
- **Find me online**
  - GitHub: github.com/barbuk
  - Web: iam.barb.uk



Roadmap

- Part 1: The Threat Landscape
- Part 2: Recent History: Package Managers & OS Repos
- Part 3: Attacks Against DevOps Tools & Developers
- Part 4: Defensive Playbook: Resolving the Update Paradox

Part 1

The Threat Landscape

Malware Packages vs. Vulnerable BOM Components

Current Situation: The Supply Chain Kill Chain

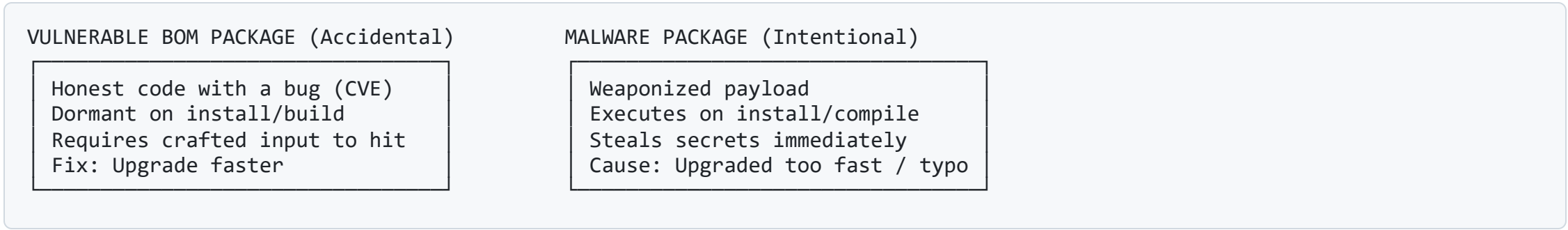
```
[ Attacker ]
     ├─> Poison Upstream (Phishing / CI Token Leak / Orphan Takeover)
     │
[ Registry / Tool ] (npm, PyPI, crates.io, AUR, VSX, GitHub Actions)
     ├─> Trusted Pull (Developer Workstation or CI/CD Runner)
     │
[ Execution ] (postinstall, build.rs, .pth, PKGBUILD, IDE Extension Load)
     ├─> Core Objectives (Harvest -> Propagate -> Persist -> Monetize)
```

- **Why the supply chain?** Hardened cloud perimeters are expensive to breach. Riding `npm install`, `cargo build`, or an IDE extension update bypasses firewalls and EDR by abusing **implicit developer trust**.
- **2025–2026 Shift:** From manual typosquatting to **automated, self-propagating worms** where one compromised token infects dozens of downstream packages in minutes.

Attacker End Goals

- **Credential & Identity Harvesting (Initial Loot)**
  - Scrape `.env` files, `~/.ssh`, `~/.aws`, browser session cookies, and high-value AI/SaaS API keys (`ANTHROPIC_API_KEY`, `OPENAI_API_KEY`).
- **Worm Self-Propagation (Supply Chain Multiplier)**
  - Steal `NPM_TOKEN`, `PYPI_API_TOKEN`, `crates.io` tokens, and GitHub PATs to automatically trojanize every package the victim maintains.
- **CI/CD & Cloud Lateral Movement**
  - Dump CI runner memory secrets to pivot into private corporate repositories, internal artifact registries, and production Kubernetes/cloud clusters.
- **Direct Financial Theft & Resource Hijacking**
  - Drain crypto wallets/exchange keys on developer laptops, resell stolen LLM/cloud quotas, or deploy covert cryptominers.
- **Long-Term Persistence & Strategic Espionage**
  - Plant stealth backdoors (`XZ Utils` `sshd`, `Claude Code` reverse SSH, `AUR` eBPF rootkits) to compromise downstream production servers.

Malware Package vs. Vulnerable BOM Component



Dimension	Vulnerable Package in BOM	Malicious Package Pulled
Trigger	Crafted runtime input (if reachable)	Install/build hook (`postinstall` , `build.rs` , `.pth`)
Time to Impact	Weeks/months	Seconds (during install or IDE index)
Detection	SBOM scanners (`grype` , `govulncheck`)	Cooldowns, lockfiles, sandboxing
Remediation	Patch or upgrade dependency	Rotate all secrets, wipe host, roll back

Part 2

Recent History (2024–2026)

Language Package Managers & OS Repositories

Language Package Managers Under Fire

- **Node.js (`npm`): Shai-Hulud / Miasma Worms (2025–2026)**
  - Self-propagating credential stealers hitting `keyv` (**61M** weekly dl), `flat-cache` (**64M** weekly dl), `cacheable` (**35M** weekly dl) — **>160M/wk combined**.
  - TanStack CI & `axios` maintainer compromises pushed trojanized builds in minutes.
- **Python (`PyPI`): Hades & `litellm` Compromises (2026)**
  - Shipped malicious `*-setup.pth` files instead of `setup.py` hooks.
  - Python executes `.pth` files automatically **on interpreter startup** (before `import`).
- **Rust (`crates.io`): `arrayref` & `internment` Poisoning (Aug 2026)**
  - Compromised maintainer injected transitive `proc-macro1` dependency.
  - `build.rs` and proc-macros execute arbitrary binaries during `cargo build` or IDE `rust-analyzer` checks.

OS & Vendor Ecosystems: AUR, Red Hat & XZ Utils

- **Arch Linux (`AUR`): "Atomic Arch" Campaign (June 2026)**
  - Attackers claimed **400+ orphaned AUR packages**: `PKGBUILD` pulled rogue npm packages (`atomic-lockfile`) dropping a **Rust infostealer** + **eBPF rootkit**.
  - **Maintainer Perspective:** AUR registrations remain closed since the attack: Arch is rolling out a **Verified AUR Developer** role to validate package adoptions and new releases.
- **Red Hat Cloud Services (`@redhat-cloud-services` May 2026 | RHSB-2026-006)**
  - Malicious VS Code extension compromised a Red Hat developer's GitHub account.
  - Pushed unauthorized commits to `RedHatInsights` repos, triggering CI to publish **32 trojanized `@redhat-cloud-services` packages** (with valid provenance) and poisoning repo configs to infect peers.
- **Linux Distros: XZ Utils (`CVE-2024-3094`)**
  - Multi-year social engineering ("Jia Tan") hid poisoned `m4` build macros inside release tarballs (absent from Git) to backdoor `sshd` via `liblzma`.

Part 3

Attacks Against DevOps Tools & Developers

CI/CD Pipelines, Hidden Runtime Layers & Social Engineering

Impact on DevOps, Cloud & IDE Tools

- **OVHcloud CLI (`v0.11.0` – `v0.15.0` Oct 2026 | @olesovhcom)**
  - Hijacked maintainer GitHub account replaced release binaries across 5 versions for 94 min with a credential-stealing binary scanning local disks for API keys.
- **Official Claude Code VSX Extension (`v2.1.312` – `v2.1.313` Oct 2026 | phorion.io)**
  - Trojanized Cursor/VSX marketplace builds injected an attacker Ed25519 SSH key (`svc-telemetry`) into `~/.ssh/authorized_keys` and opened a reverse SSH tunnel on editor startup.
- **Aqua Security Trivy (`v0.69.4` & `trivy-action` Mar 2026)**
  - Attackers poisoned the security scanner binary and repointed **76 of 77 Git tags** in `aquasecurity/trivy-action` to exfiltrate AWS/GCP/K8s secrets during CI scans.
- **Open VSX "GlassWorm" (2025–2026)**
  - Self-propagating VSX worm (code formatters / time trackers) using **Solana, P2P, and Google Calendar** as dead-drop C2s.

Compromising CI/CD: GitHub Actions

Case Study: `tj-actions/changed-files` (`CVE-2025-30066`)

- **Pivot:** Leaked token from `reviewdog/action-setup` (`CVE-2025-30154`).
- **Tag Hijack:** Attackers retroactively repointed mutable version tags (`v1`, `v2`, `v45`) across 23,000+ repositories to a malicious commit.
- **Exfiltration:** Payload scraped CI runner memory and printed base64-encoded AWS keys, npm tokens, and PATs into public workflow logs.

GitLab RCE & The Alpine Linux Incident

- **The Flaw:** `CVE-2026-110264` & `CVE-2026-97229` (CVSS 10.0 Unauthenticated RCE).
- **Root Cause:** Integer overflow in **Onigmo** (Ruby's C regex engine), not GitLab's Ruby application logic.

```
[ GitLab Application: Updated & Patched ] ──calls──► [ System Ruby / Onigmo Regex: UNPATCHED! ]
```

- Omnibus GitLab bundles patched Ruby: Alpine's package used **system Ruby**.
- Attackers probed Alpine's instance (Oct 20–21, 2026), crashing workers.
- **Response:** Alpine assumed breach, rebuilt infrastructure from scratch, and rotated all credentials/tokens/2FA.

Vendored C Debt & Targeting Developers

Broccoli2Shell (`CVE-2026-111497` | broccoli2shell.com)

- Heap out-of-bounds write in Google's C **Brotli** decoder.
- Fix landed in Google Brotli source in 02/2026 without a security advisory or new release.
- Statically copied inside Node.js, Python bindings, RubyGems, Go modules, .NET runtime, Varnish, and Palo Alto PAN-OS.
- Updating host `libbrotli` does not patch runtimes embedding old C copies.

Targeting Developers (Rust Security Advisory)

- State-sponsored actors (DPRK-style) target maintainers via fake recruiter outreach and coding assessments.
- Cloning and building the "interview repo" compromises the workstation and steals `crates.io` / `npm` publishing keys.

Part 4

Defensive Playbook

Resolving the Update Paradox for You & Your Team

The Update Paradox & Dependency Cooldowns



- **Dependency Cooldowns (cooldowns.dev):** Ignore uncurated releases newer than 3–7 days.
  - **Pros:** Registry worms and hijacked releases are detected and yanked within **24–72h**.
  - **Cons:** Delays routine bugfixes: requires an explicit bypass path for urgent CVE patches.

```
# .npmrc (npm >= 11.10)          # uv.toml                    # renovate.json
min-release-age=7                exclude-newer = "7 days"     { "minimumReleaseAge": "7 days" }
```

Why Trusted OS Security Channels Should Bypass Cooldowns

Property	Language Registry (`npm`, `PyPI`, `crates.io`)	Official OS Security Repo (`debian-security`, RHEL)
Publishing Model	Push-to-world by individual maintainer token	Curated by dedicated Distro Security Team
Update Scope	Full version bump (new code + new dependencies)	**Surgical backport** of minimal CVE patch only
Build & Trust	Often built on developer laptop or GitHub runner	Built from source on isolated buildds + **GPG-signed** (`InRelease`)
Recommended Policy	**Enforce 3–7 day cooldown**	**Patch immediately (`unattended-upgrades`)**

Pin Everything: CI, Hooks & Lockfiles

- **GitHub Actions** `pinact`: Pin mutable tags (`@v4`) to full commit SHAs.

```
- uses: actions/checkout@de0fac2e4500dabe0009e67214ff5f5447ce83dd # v4.2.2
```

- **Git Hooks** `prek`: Fast Rust drop-in for `pre-commit`: pin hook revisions to immutable commits:

```
prek autoupdate --freeze --cooldown-days 10
```

- **Enforce Lockfiles in CI & Use OIDC Trusted Publishing:**
  - Never run bare `npm install` or `cargo build` in CI: enforce frozen lockfiles:

```
npm ci --ignore-scripts   # Node
cargo build --locked      # Rust
uv sync --frozen          # Python
```

  - Replace long-lived `npm` / `PyPI` / `crates.io` tokens with short-lived **OIDC Trusted Publishing**.

Container Tag Strategy: Match Your Team Maturity

Organization Setup	Recommended Strategy	Why
Dedicated SecOps Team *(tracks SBOMs, lifecycles, CVEs, and EOLs daily)*	**Pin specific tag / SHA256 digest** (`alpine:3.21.2@sha256:...`)	Full reproducibility: SecOps actively triggers rebuilds when base OS libraries (e.g., Ruby or `libbrotli`) patch CVEs.
No Dedicated SecOps Team *(small dev team, no lifecycle tracking)*	**Use `latest` or a stable tag + strict security monitoring**	Pinning an exact tag when nobody tracks EOLs freezes unpatched OS libraries in production forever.

Sandbox Language Builds & Clean Chroot OS Packaging

- **Isolate Language Builds (`bubblewrap`, `podman`, `docker`):**
  - Hide `~/.ssh`, `~/.aws`, and browser profiles during `npm install` or `cargo build`:

```
bwrap --ro-bind /usr /usr --ro-bind /lib /lib --ro-bind /lib64 /lib64 \
      --bind "$PWD" /work --chdir /work --unshare-all --share-net npm ci --ignore-scripts
```

- **OS Packaging in a Clean Chroot (Arch `pkgctl build` / `makechrootpkg`, Debian `sbuild` / `pbuilder`):**
  - **Host & Secret Isolation:** Upstream `Makefile` / `PKGBUILD` cannot read `~/.ssh`, GPG signing keys, or `$HOME`.
  - **Offline Build Phase:** Sources are fetched and hash-verified *first*: compilation runs with network disabled (`--private-network`).
  - **Reproducibility & Zero Contamination:** Debian leads with **98%+** reproducible packages: Arch Linux reaches **94%** in `core` (**80%** overall).

Secret Scanning & Continuous SBOMs

1. **Understand & Audit Exposed Secrets:**
   - `trufflehog`: Scans Git history, CI logs, and filesystems: verifies if leaked credentials are actively live.
   - `gitleaks`: Fast regex/entropy scanner wired into local `prek` hooks and CI pipelines.
2. **Maintain & Query SBOMs Continuously:**
   - `grype`: Scan container images, directories, and SBOMs (`grype sbom:sbom.json`) for OS and library CVEs.
   - `Renovate`: Automate dependency updates with built-in cooldown rules and digest pinning.
3. **Language-Native Reachability Analysis:**
   - `govulncheck ./...`, `npm audit fix`...

See Something, Say Something

- **Humans Catch What Scanners Miss:**
  - **XZ Utils:** Caught because one engineer investigated a **500ms `sshd` latency spike**.
  - **Alpine GitLab:** Caught because ops investigated **Sidekiq worker crashes** instead of blindly restarting.
  - **OVHcloud CLI:** Caught in **94 minutes** because a user reported a binary hash mismatch.
- **Building the Culture:**
  - **Zero-Blame Early Reporting:** Just like phishing: if a dev runs a trojanized repo or suspicious package, fast self-reporting limits blast radius to minutes: blame guarantees cover-ups.
  - **Low-Friction Security Channel:** Keep a noisy-friendly `#security-radar` channel to flag weird `postinstall` output, checksum mismatches, or upstream advisories.
  - **Don't Just Re-Run Failed Builds:** Treat unexpected hash changes or CI crashes as signals to inspect, not annoyances to bypass.

Mitigating the Impact of an Infection (Blast-Radius Control)

- **Shrink the Loot:**
  - **Hardware-Backed Keys (`FIDO2` / YubiKey):** Use `ed25519-sk` SSH keys and YubiKey GPG keys requiring physical touch: infostealers cannot copy private keys that never touch disk.
  - **Ephemeral Tokens & Least-Privilege CI:** Replace long-lived tokens with short-lived **OIDC** (15m TTL), fine-grained PATs, and `permissions: contents: read` on ephemeral runners.
- **When Infection Strikes:**
  - **Revoke Everything Immediately:** Uninstalling the malware package fixes nothing—rotate all cloud IAM keys, PATs, registry tokens, and browser session cookies.
  - **Audit Persistence & Downstream Releases:** Check `~/.ssh/authorized_keys`, cron/systemd units, and verify no worm commits/releases were pushed by the victim's account.
  - **Wipe & Rebuild:** Never "clean" a compromised workstation or runner: nuke and rebuild from scratch.

Next Monday Morning Checklist

- **Set a 7-day cooldown** in `.npmrc`, `uv.toml`, and `renovate.json` (see cooldowns.dev).
- **Pin CI, hooks & lockfiles** with `pinact run`, `prek`, and `--locked` / `npm ci`.
- **Build in sandboxes / clean chroots** (`bubblewrap`, `podman`, `pkgctl build`, `sbuild`).
- **Scan secrets** in hooks and CI using `gitleaks` and `trufflehog`.
- **Automate SBOM & reachability checks** with `grype` or `trivy` + `renovate`.
- **Check your OS EOL & security updates:** `unattended-upgrade --dry-run -v`